

Dart Programming

An Introduction to Flutter

Lesson 1:



INTRODUCING FLUTTER

- Flutter is Google's portable UI framework for building modern, native, and reactive applications for iOS and Android.
- Flutter is an open-source project hosted on GitHub with contributions from Google and the community.
- Flutter uses Dart, a modern object-oriented language that compiles to native ARM code and production-ready JavaScript code.
- Flutter uses the Skia 2D rendering engine that works with different types of hardware and software platforms and is also used by Google Chrome, Chrome OS, Android, Mozilla Firefox, Firefox OS, and others.
- Skia is sponsored and managed by Google and is available for anyone to use under the BSD Free Software License.
- Skia uses a CPU-based path render and also supports the OpenGL ES2-accelerated backend.

INTRODUCING FLUTTER

- Flutter uses Dart to create your user interface, removing the need to use separate languages like Markup or visual designers.
- Flutter is declarative; in other words, Flutter builds the UI to reflect the state of the app. When the state (data) changes, the UI is redrawn, and Flutter constructs a new instance of the widget.
- Flutter is fast, and the rendering runs at 60 frames per second (fps) and 120fps for capable devices.
- The higher the fps, the smoother the animations and transitions.
- Applications made in Flutter are built from a single codebase, are compiled to native ARM code, use the graphics processing unit (GPU), and can access specific iOS and Android APIs (like GPS location, image library) by communicating via platform channels.

Defining Widgets and Elements

- The Flutter UI is implemented by using widgets from a modern reactive framework. Flutter uses its own rendering engine to draw widgets.
- Widgets are the building blocks of a Flutter app, and each widget is an immutable declaration of the user interface.
- In other words, widgets are configurations (instructions) for different parts of the UI.
- Placing the widgets together creates the widget tree.
- Flutter uses the widget as the configuration to build each element, which means the element is the widget that is mounted (rendered) on the screen.
- The elements that are mounted on the screen create the element tree.

Defining Widgets and Elements

- Widgets with structuring elements such as a list, grid, text, and button
- Widgets with input elements such as a form, form fields, and keyboard listeners
- Widgets with styling elements such as font type, size, weight, color, border, and shadow
- Widgets to lay out the UI such as row, column, stack, centering, and padding
- Widgets with interactive elements that respond to touch, gestures, dragging, and dismissible
- Widgets with animation and motion elements such as hero animation, animated container, animated crossfade, fade transition, rotation, scale, size, slide, and opacity
- Widgets with elements like assets, images, and icons
- Widgets that can be nested together to create the UI needed
- Custom widgets you can create yourself

UNDERSTANDING WIDGET LIFECYCLE EVENTS

- To build the UI, you use two main types of widgets, **StatelessWidget** and **StatefulWidget**.
- A stateless widget is used when the values (state) do not change, and the stateful widget is used when values (state) change.
- Each stateless or stateful widget has a **build** method with a **BuildContext** that handles the location of the widget in the widget tree.
- The **BuildContext** objects are actually Element objects, an instantiation of the Widget at a location in the tree.

The StatelessWidget Lifecycle

- A **StatelessWidget** is built based on its own configuration and does not change dynamically.
- For example, the screen displays an image with a description and will not change. The stateless widget is declared with one class.
- The build (the UI portions) method of the stateless widget can be called from three different scenarios.
- It can be called the first time the widget is created, when the widget's parent changes, and when an **InheritedWidget** has changed.
 - class JournalList extends StatelessWidget {
 - @override
 - Widget build(BuildContext context) {
 - return Container();
 - }
 - }

The StatefulWidget Lifecycle

- A **StatefulWidget** is built based on its own configuration but can change dynamically.
- For example, the screen displays an icon with a description, but values can change based on the user's interaction, like choosing a different icon or description.
- This type of widget has a mutable state that can change over time.
- The stateful widget is declared with two classes, the **StatefulWidget** class and the **State** class.
- The **StatefulWidget** class is rebuilt when the widget's configuration changes, but the **State** class can persist (remain), enhancing performance.
- If the **StatefulWidget** is removed from the tree and then inserted back into the tree sometime in the future, a new **State** object is created.
- You call the `setState()` method to notify the framework that this object has changes, and the widget's `build` method is called, we would set the new state values inside the `setState()` method.

The StatefulWidget Lifecycle

- The following example shows a StatefulWidget base structure, we have two classes, the JournalEdit StatefulWidget class and the _JournalEditState class.
 - class JournalEdit extends StatefulWidget {
 - @override
 - _JournalEditState createState() => _JournalEditState();
 - }
 - class _JournalEditState extends State<JournalEdit> {
 - @override
 - Widget build(BuildContext context) {
 - return Container();
 - }
 - }

UNDERSTANDING THE WIDGET TREE AND THE ELEMENT TREE

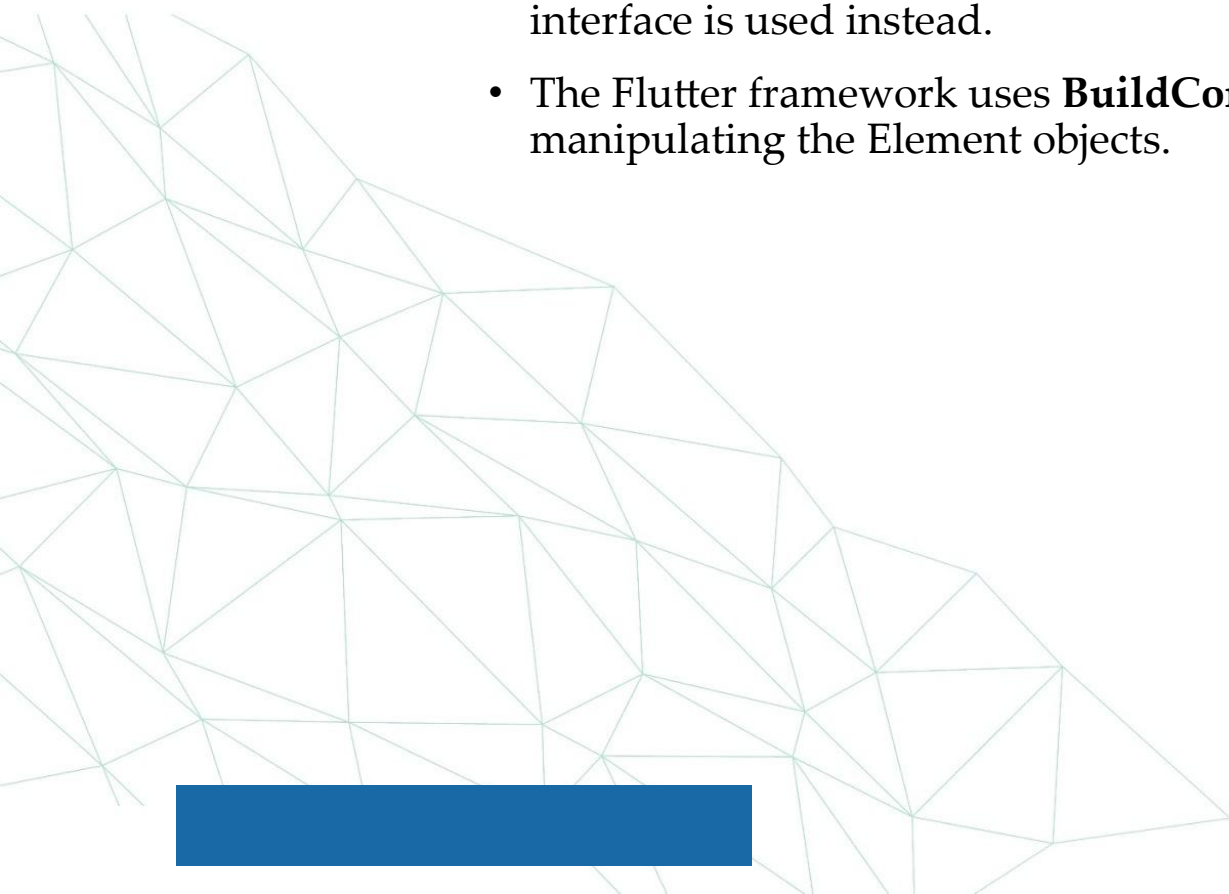
- The Flutter framework uses the widgets as the configurations for each element that is mounted (rendered) on the screen.
- The mounted elements displayed on the screen create the element tree.
- We now have two trees, the widget tree that has the widget configurations and the element tree that represents the rendered widgets on the screen.
- When the application starts, the `main()` function calls the **`runApp()`** method, usually taking a **`StatelessWidget`** as the argument, meaning that the main application itself is a widget, and is mounted as the root element for the application.
- The Flutter framework processes through all of the widgets, and each corresponding element is mounted.

UNDERSTANDING THE WIDGET TREE AND THE ELEMENT TREE

- As we can see, just about everything in Flutter is a widget.
 - `void main() => runApp(MyApp());`
 - `class MyApp extends StatelessWidget {`
 - `@override`
 - `Widget build(BuildContext context) {`
 - `return MaterialApp(`
 - `title: 'Flutter App',`
 - `theme: ThemeData(`
 - `primarySwatch: Colors.blue,`
 - `),`
 - `home: MyHomePage(title: 'Home'),`
 - `);`
 - `}`
 - `}`

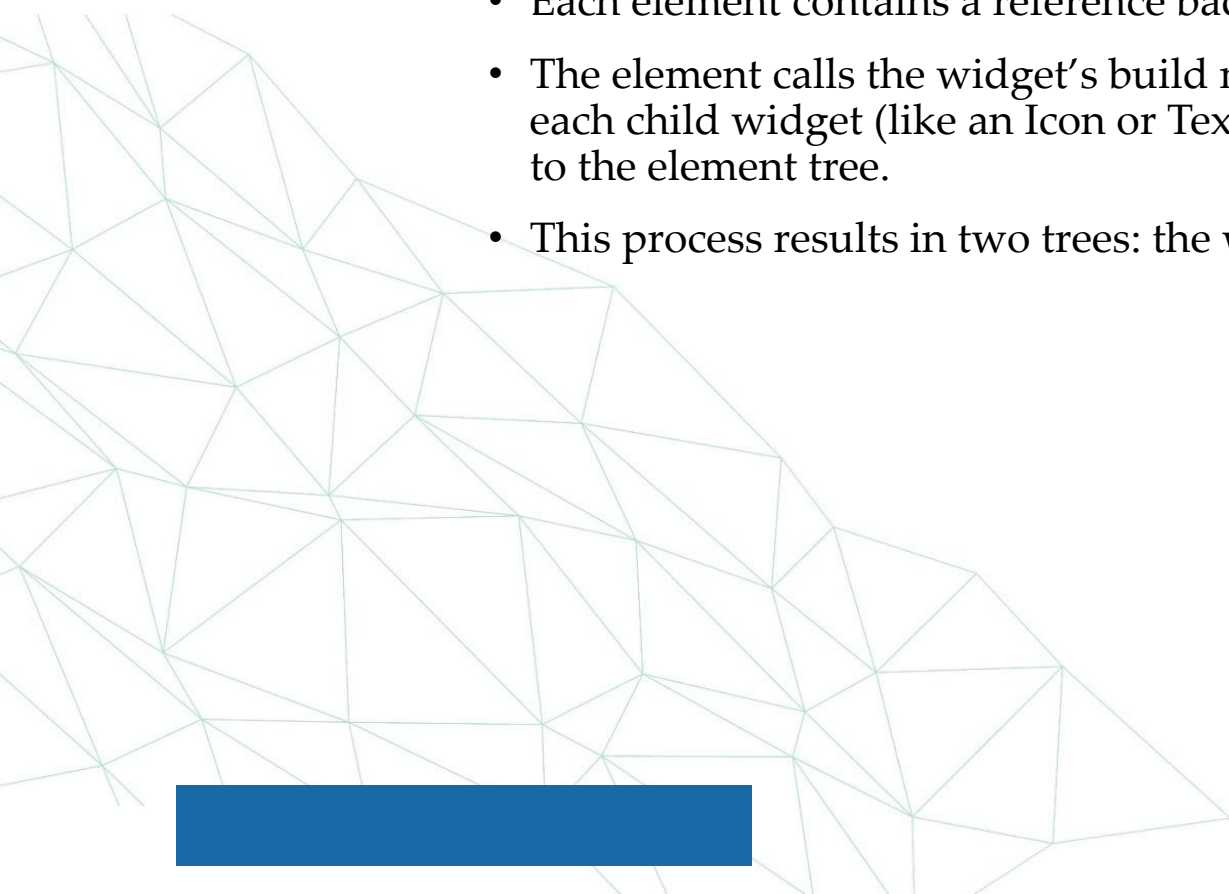
UNDERSTANDING THE WIDGET TREE AND THE ELEMENT TREE

- Elements have a reference to the widget and are responsible for comparing the widget differences.
- If a widget is responsible for building child widgets, then elements are created for each child widget.
- To discourage direct manipulation of the Element objects, the **BuildContext** interface is used instead.
- The Flutter framework uses **BuildContext** objects to discourage you from manipulating the Element objects.



Stateless Widget and Element Trees

- A stateless widget has the configuration to create a stateless element.
- Each stateless widget has a corresponding stateless element.
- Flutter framework makes a request from the widget to create an element and then mounts (adds) the element to the element tree.
- Each element contains a reference back to the widget.
- The element calls the widget's build method to check for children widgets, and each child widget (like an Icon or Text) creates its own element and is mounted to the element tree.
- This process results in two trees: the widget tree and the element tree.



Stateful Widget and Element Trees

- A stateful widget has the configuration to create a stateful element.
- Each stateful widget has a corresponding stateful element.
- The Flutter framework calls the **createElement** method to create the stateful element, and the stateful element is mounted to the element tree.
- Since this is a stateful widget, the stateful element requests that the widget create a state object by calling the **StatefulWidget** class's **createState** method.
- The stateful element now has a reference to the state object and the widget at the given location in the element tree.
- The stateful element calls the state object widget's **build** method to check for child widgets, and each child widget creates its own element and is mounted to the element tree.
- This process results in two trees: the widget tree and the element tree.
- The state object maintains a reference to the widget (**StatefulWidget** class) and also handles the construction for the **Text** widget with the latest value.

Stateful Widget and Element Trees

- A stateful widget has the configuration to create a stateful element.
- Each stateful widget has a corresponding stateful element.
- The Flutter framework calls the **createElement** method to create the stateful element, and the stateful element is mounted to the element tree.
- Since this is a stateful widget, the stateful element requests that the widget create a state object by calling the **StatefulWidget** class's **createState** method.
- The stateful element now has a reference to the state object and the widget at the given location in the element tree.
- The stateful element calls the state object widget's **build** method to check for child widgets, and each child widget creates its own element and is mounted to the element tree.
- This process results in two trees: the widget tree and the element tree.
- The state object maintains a reference to the widget (**StatefulWidget** class) and also handles the construction for the **Text** widget with the latest value.

Stateful Widget and Element Trees

- To set new data (properties/variables) values, call the `setState()` method to update the state object, and the state object marks the element as dirty (has changed) and causes the UI to be updated (scheduled).
- The stateful element calls the state object's `build` method to rebuild the children widgets.
- A new widget is created with the new state value, and the old widget is removed.
 - `class JournalEdit extends StatefulWidget {`
 - `@override`
 - `_JournalEditState createState() => _JournalEditState();`
 - `class _JournalEditState extends State<JournalEdit> {`
 - `String note = 'Trip A';`
 - `void _onPressed() {`
 - `setState(() {`
 - `note = 'Trip B';`
 - `});`

Stateful Widget and Element Trees

- @override
- Widget build(BuildContext context) {
- return Column(
• children: <Widget>[
• Icon(),
• Text('\$note'),
• FlatButton(
• onPressed: _onPressed,
•),
•],
•);
• }
• }

INSTALLING THE FLUTTER SDK

- Installing on Windows:
 - System Requirements:
 - Windows 7 SP1 or later (64-bit)
 - 400 MB of disk space (does not include disk space for a integrated development environment and other tools)
 - The following command-line tools:
 - PowerShell 5.0 or newer
 - Git for Windows (Git from the Windows Command Prompt)
 - Get the Flutter SDK
 - Download the installation file from
 - <https://flutter.dev/docs/get-started/install/windows/>
 - Extract the file in the desired location. Do not install Flutter in directories that require elevated privileges like C:\Program Files\.
 - Look in the extracted directory\flutter and double-click the flutter_console.bat file.

INSTALLING THE FLUTTER SDK

- Update the path:
 - Open Control Panel and drill down to Desktop App ⇔ User Accounts ⇔ User Accounts ⇔ Change My Environment Variables.
 - Under User Variables for WindowsUserName, select the Path variable and click the Edit button
 - In the Edit environment variable, click the New button.
 - Type the path C:\Users\WindowsUserName\flutter\bin.
 - Click the OK button and then close the Environment Variables screen.
 - Reboot Windows to apply the changes.
- Check for Dependencies
 - Run the following command from the Windows command prompt to check for dependencies that need to be installed to finish the setup:
 - flutter doctor
 - View the report and check for other software that may need to be installed.

Install Android Studio

- Download and install Android Studio from <https://developer.android.com/studio/>.
- Start Android Studio and follow the Setup Wizard, which installs all of the Android SDKs required by Flutter.
- Set Up the Android Emulator:
 - Enable VM acceleration on your machine.
 - Currently, Android Studio requires an open Android project before you can access the Android submenu.
 - From Android Studio, select Tools ⇨ Android ⇨ AVD Manager ⇨ Create Virtual Device.
 - Select a device of your choice and click Next.
 - Select an x86 or x86_64 image for the Android version to emulate.

Install Android Studio

- If available, make sure you select Hardware – GLES 2.0 to enable hardware acceleration.
- This hardware acceleration will make the Android emulator run faster.
- Click the Finish button.
- To check that the emulator image has been installed correctly, select Tools
⇒ AVD Manager and then click the Run button (play icon).

Flutter compilation (Dart)

- **Development compilation:**

- During development, Flutter uses JIT compilation in development mode. This enables important development features such as hot reload. Due to the power of Dart's compiler, interactions between the code and the simulator/device are really fast, and debugging info helps developers step into the source code.

- **Release compilation**

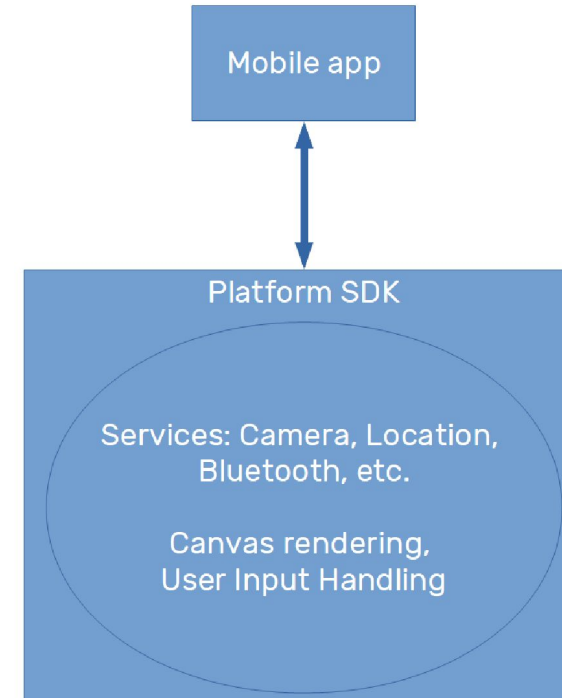
- In release mode, debugging information is not necessary, and the focus is performance.
- Flutter uses a technique that's common to game engines. By using AOT mode, Dart code is compiled to native code, and the app loads the Flutter library and delegates rendering, input, and event handling to it by using the Skia engine.

Supported platforms

- By now, Flutter supports ARM Android devices running at least on Jelly Bean 4.1.x version, and iOS devices from iPhone 4S or newer.
- Of course, Flutter apps can normally be run on simulators.
- Google is intending to port the Flutter runtime to the web by using the Dart capability of compiling to JavaScript.
- Initially called **Hummingbird**, this project now is known as "Flutter for web".

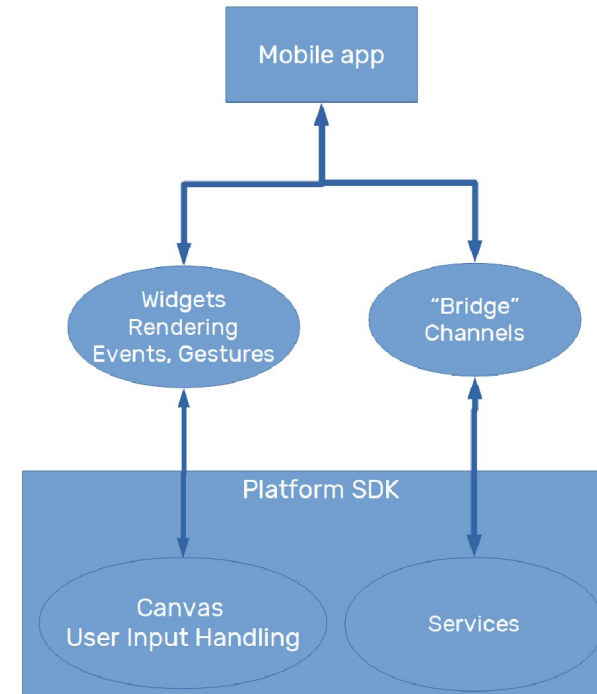
Flutter rendering

- One of the main aspects that makes Flutter unique is the way that it draws the visual components to the screen.
- The big difference is how the application communicates with the platform's SDK, what it asks the SDK to do, and what it does by itself:
- The platform SDK can be seen as the interface between applications and the operation system and services.
- Each system provides its own SDK with its own capabilities and is based on a programming language (that is, Kotlin/Java for the Android SDK and Swift/Objective C for the iOS SDK).



Flutter – rendering by itself

- Flutter chooses to do all the hard work by itself. The only thing it needs from the platform's SDK is access to **Services** APIs and a canvas to draw the UI on:
- Flutter moves the widgets and rendering to the app, from where it gets the customization and extensibility.
- Through a canvas, it can draw anything and also access events to handle user inputs and gestures by itself.
- The bridge in Flutter is done by platform channels.



Hello Flutter

- With the Flutter development environment configured create a new Flutter Project.
- A typical Flutter project structure created
- Listing the basic structure elements, we get the following:
 - **android/ios:** This contains the platform-specific codes. If you already know the Android project structure from Android Studio, there is no surprise here.
 - The same goes for XCode iOS projects.
 - **hello_flutter.iml:** This is a typical IntelliJ project file containing the `JAVA_MODULE` information used by the IDE.
 - **lib directory:** This is the main folder of a Flutter application; the generated project should contain at least a `main.dart` file to start work on.

Name	Size
.gitignore	1 KB
▶ .idea	--
.metadata	303 bytes
.packages	2 KB
▶ android	--
hello_flutter.iml	896 bytes
▶ ios	--
▶ lib	--
pubspec.lock	3 KB
pubspec.yaml	2 KB
README.md	542 bytes
▶ test	--

Hello Flutter

- `pubspec.yaml` and `pubspec.lock`: this `pubspec.yaml` file is what defines a Dart package, and this is one of the main files of the project, where you list the application dependencies and in Flutter's case more than that.
- `README.md`: This file typically has a description of the project, and it is very common in open source projects.
- `test` directory: This contains all the test-related files of the project. Here, we can add unit tests, and also widget tests by using Flutter specific-packages.

pubspec file

- The **pubspec** file in Flutter is similar to a simple Dart package. Besides that, it contains an additional section for configurations specific to Flutter. Let's see the **pubspec.yaml** file's contents in details:
 - we have the **dependencies** section of the **pubspec** file:
 - It starts with the **environment** property with the Dart SDK version constraints defined.
 - Then it has the **dependencies** property, which starts with the main dependency of a Flutter application, the Flutter SDK itself, which contains many of Flutter's core packages.
 - As an additional dependency, the generator adds the **cupertino_icons** package, which contains icon assets used by the built-in Flutter Cupertino widgets

pubspec file

- The `dev_dependencies` property contains only the `flutter_test` package dependency provided by the Flutter SDK itself, and contains Flutter-specific extensions to the already-known Dart `test` package.
- The `flutter` section allows us to configure resources that are bundled in the application to be used during runtime, such as **images**, a **fonts**, and a JSON file, typically, any nonsource code file that helps in the app's composition:
 - **uses-material-design**
 - **assets**: This property is used to list the resource paths that will be bundled with the final application. The **assets** files can be organized in any way; what matters for Flutter is the path of the files. You specify the path of the file relative to the project's root. This is used later in Dart code when you need to refer to an asset file.